ARadhana: A Geofence based Augmented Reality navigation and Game Platform for Heritage Exploration

Sanket Shrestha
Sunway College Kathmandu
sanket_24128469@sunway.edu.np

Prashant Adhikari
Sunway College Kathmandu
prashant_adhikari_a25@sunway.edu.np

Rishu Prajapati
Sunway College Kathmandu
rishu_prajapati_a25@sunway.edu.np

Orchid HackX 2026 · Team Valhalla · July 11, 2026

Executive Summary

Navigation for visitors in Nepal typically ends at the entrance of a heritage site or campus: beyond the gate there is no structured guidance, no contextual storytelling, and no way for an institution to know whether anyone actually explored what lies inside. ARadhana closes that gap with a browser-based platform that turns a visit into a guided, verifiable, and rewarding experience. A visitor picks a mapped route and is steered from checkpoint to checkpoint by a live compass and an augmented-reality camera view; each checkpoint is protected by a geofence (a configurable radius, 10 m by default) so location-specific videos and stories unlock only when the visitor is physically present and confirms arrival. Exploration is gamified end to end: milestones and side quests pay out private AR Points, completing a route earns a permanent badge and unlocks a five-question quiz, points can be redeemed for tangible campus perks, and a community leaderboard ranks explorers by milestones, places, and side quests. Institutions manage everything without code — routes, geofence radii, quizzes, users, and engagement analytics — through a separate administrator dashboard. The complete system was built and field-tested at Orchid College in Kathmandu during Orchid HackX 2026: real users completed the seeded campus routes, earned badges and points, and appeared on the leaderboard, demonstrating a working vertical slice that can be pointed at any campus, museum, or heritage site with nothing more than an admin account and a set of coordinates.

Keywords: augmented reality, geofencing, GPS navigation, gamification, cultural heritage, campus exploration, web AR, Nepal tourism

Contents

Chapter 1

Introduction

1.1 Background

Nepal's cultural landscape is dense with sites worth exploring: the country holds ten UNESCO World Heritage properties, four of them cultural sites within the Kathmandu Valley alone, and it welcomed 1.147 million international tourists in 2024. Yet the experience of actually visiting such a site — or, at a smaller scale, of a newcomer finding their way around a university campus — remains largely unassisted. Navigation applications reliably bring a visitor to the entrance of a monument or an institution, but the guidance ends at the gate. Inside, visitors depend on scarce physical tourism centres, sparse signage, or the availability of a human guide.

At the same time, the technical preconditions for a software answer to this gap have matured. Modern smartphones ship with GPS receivers, magnetometers, and cameras as standard hardware, and web browsers now expose geolocation, device orientation, and camera access directly to web pages, making location-based Augmented Reality (AR) possible without installing a native application. ARadhana was built on this premise for Orchid HackX 2026 by Team Valhalla: a guided, gamified AR walk that begins where conventional navigation stops.

1.2 Project Overview

ARadhana is a mobile-first, gamified AR navigation platform for educational campuses and heritage sites. A user opens the web application in a smartphone browser, selects a pre-mapped route, and is guided in real time by a live compass arrow together with a GPS distance readout to the next waypoint. When the user physically arrives at a checkpoint, the system detects their presence through geofencing and confirms the visit; only then is the location-specific media for that checkpoint unlocked — a video testimonial, a historical image, or a narrative clip — rendered anchored in the real world through the device camera using an AR overlay.

Exploration is reinforced by a gamification layer. Users accumulate AR Points through quiz answers, milestone completions, and side quests; earn digital milestone badges for completing all milestone stops of a route; answer five-question multiple-choice knowledge quizzes unlocked after finishing a route; and compete on a community leaderboard. Points are redeemable for tangible campus perks through a reward store, and optional side quests reward detours that require a minimum five-minute on-site stay.

The platform is administered without code. An admin dashboard provides a waypoint logger for defin-

ing route coordinates, waypoint types, and media; a place management interface for creating, updating, and deleting visiting places together with their cascaded route data; quiz management; user management; and aggregate statistics. Because each place records a visitor count as visits are confirmed, the system generates engagement analytics organically, with no additional instrumentation.

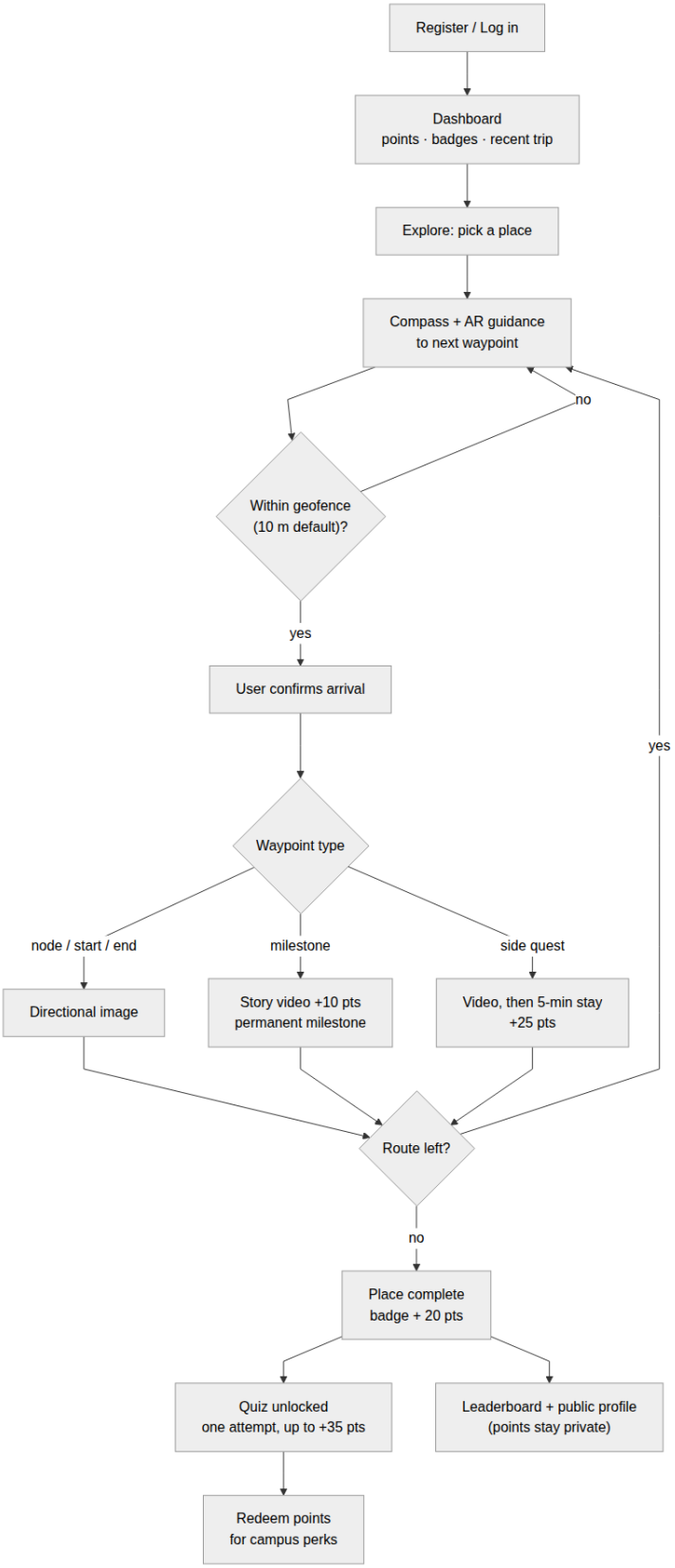

Figure 1.1: End-to-end user journey: from registration to reward redemption.

1.3 Scope

This build delivers a complete vertical slice of the platform as a web application, proven end-to-end on a single campus route at Orchid College in Kathmandu. In scope are: route selection and real-time compass navigation; geofence-verified checkpoint confirmation; AR media playback anchored in the camera view; the full gamification loop (AR Points, milestones, side quests, quizzes, badges, leaderboard, and reward redemption); and the complete admin tooling required to configure routes, media, quizzes, and users.

Explicitly out of scope for this build are native mobile applications (the product is browser-based only), offline operation (an active network connection is required for the real-time proximity stream and media delivery), and multi-language support (the interface is English-only). The same infrastructure is designed to be pointed at other premises — heritage sites such as Pashupatinath or Boudhanath — but only the Orchid College route is demonstrated here.

1.4 Report Organisation

The remainder of this report is organised as follows.

- Chapter 2 states the problem the platform addresses and the motivation for solving it now.
- Chapter 3 lists the general and specific objectives of the project.
- Chapter 4 summarises the market context, competitive landscape, and target user segments.
- Chapter 5 summarises the design process that shaped the product.
- Chapter 6 describes the overall system architecture, from the browser client to the clustered backend.
- Chapter 7 documents the database schema and the relationships between its collections.
- Chapter 8 details the gamification engine: points, milestones, side quests, quizzes, badges, and rewards.
- Chapter 9 explains the AR pathfinder — geofencing, the compass HUD, and the camera overlay.
- Chapter 10 justifies the technology stack used across all layers.
- Chapter 11 presents the results achieved during the hackathon build.
- Chapter 12 discusses current limitations and future work.
- Chapter 13 concludes the report.
- Appendix A provides the API reference, Appendix B the field-level database schema reference, and Appendix C selected code listings.

Chapter 2

Problem Statement

The problem this project addresses can be stated in a single sentence:

> Existing navigation systems stop at the entrance of heritage sites, leaving visitors without guidance to explore monuments, discover historical significance, or experience Nepal's cultural heritage in a structured and engaging way.

2.1 The Situation

Nepal welcomed 1.147 million international tourists in 2024,[1] and its heritage sites collectively receive over a million visitors annually. Consumer navigation applications serve these visitors well up to a point: they can route anyone to the gate of a temple complex, a museum, or a university campus. Beyond the gate, however, the visitor is on their own. Physical tourism centres — which provide maps, information about places of interest, and basic visitor services — have been established in Nepal only in limited places, a scarcity identified in the literature as a serious deterrent to the development of tourism in the country.[2] The same pattern holds at a smaller scale on educational campuses, where newcomers face unfamiliar geography with no structured way to explore it.

2.2 The Specific Gap

Once inside a site, four things are missing.

1. **No structured navigation.** There is no guided route through the premises; visitors wander, miss significant locations, and have no sense of a complete journey.
2. **No contextual storytelling.** The history and significance of each monument or location is not delivered at the place itself, where it would be most meaningful.
3. **No engagement incentive.** Nothing rewards deeper exploration, revisits, or learning; the visit is passive.
4. **No verification of presence.** Institutions have no way to confirm that a visitor actually stood at a location, which rules out trustworthy completion records, rewards, or engagement analytics.

This gap is not merely anecdotal. UNESCO recommends that every heritage site provide visitors with easily accessible information — maps, navigation, and cultural interpretation — through a centralised platform, while observing that surprisingly few World Heritage sites actually provide such accessible digital resources.[3] UN Tourism likewise emphasises that digital technologies play a vital role in improving cultural tourism by modernising heritage interpretation, increasing visitor engagement, and delivering innovative educational experiences.[4]

[1] https://tourism.gov.np/content/83/nepal-tourism-statistics-2024/

[2] https://andjournal.in/2018/07/13/tourism-importance-prospects-and-challenges-with-special-reference-to-nepal/

[3] https://whc.unesco.org/en/sustainabletourismtoolkit/guide5/

[4] https://webunwto.s3-eu-west-1.amazonaws.com/imported_images/50679/unwto_unesco_istanbul_conf_concept_note_22.11.2018_en.pdf

2.3 Why Existing Approaches Fall Short

The traditional answer to the guidance gap is the human-guided tour — the campus orientation walk or the hired heritage guide. This model has four structural weaknesses:

- **Passive and forgettable.** A guide speaks, the audience listens, and little is retained.
- **Non-scalable.** Every tour requires a human guide and cannot run simultaneously for hundreds of newcomers.
- **Zero data.** Institutions gain no insight into which locations visitors actually engage with.
- **One-and-done.** There is no replay value and no incentive to revisit.

A number of digital alternatives exist in Nepal — curated guide applications, archival heritage databases, self-guided GPS tour products, and QR-coded storyboards installed at sites — but none combines real-time wayfinding with AR presentation, presence verification, and gamified engagement, and some depend on physical hardware installed and maintained at every location. A detailed comparison of these competitors is given in Chapter 4.

2.4 Consequence of Leaving It Unsolved

If the gap remains open, the cost is borne on all sides. Visitors leave heritage sites having seen stone and brick but not the stories that give them meaning; students finish orientation without ever genuinely learning their campus. Institutions continue to pay for guided tours that do not scale and produce no data, and they miss the engagement and educational outcomes that international tourism bodies explicitly identify as the value of digital interpretation. For Nepal in particular — a country whose tourism sector depends heavily on its cultural heritage — every undirected, unengaged visit is an opportunity lost.

2.5 Motivation

Two developments make this problem solvable now rather than at some future date. First, smartphones are ubiquitous among both tourists and students, and each one already carries the required hardware: a GPS receiver, a magnetometer, and a camera. Second, browser platform APIs for geolocation, device orientation, and camera access have matured to the point where location-based AR runs directly in a mobile web page, with no application store, no installation step, and no physical infrastructure at the site. A solution can therefore be deployed to any premises purely as software.

The project deliberately starts with a campus as its proving ground. A campus is a controlled premises: routes are short, waypoints are known, users are reachable for feedback, and the complete loop — geofenced navigation, AR media, gamification, and admin tooling — can be validated end-to-end within the hackathon timeframe. Once proven at Orchid College, the same configurable platform can be pointed at heritage sites such as Pashupatinath, Boudhanath, or any of Nepal's ten UNESCO World Heritage properties without changes to the codebase.

Chapter 3

Objectives

3.1 General Objective

The general objective of this project is to build a self-guided, geofence-verified AR exploration platform that makes heritage and campus visits engaging, measurable, and scalable — replacing the human-guided tour with software that guides visitors in real time, verifies their physical presence, and rewards exploration.

3.2 Specific Objectives

To realise the general objective, ARadhana pursues the following specific objectives:

- **Verify physical presence via dynamic geofencing.** Detect and confirm a visitor's arrival at each waypoint using a geofence radius of 10 metres stored per place, so that visits, rewards, and analytics are grounded in verified presence rather than self-reporting.
- **Guide visitors in real time.** Provide continuous wayfinding to the next waypoint through a live compass heads-up display with a GPS distance readout, together with an AR camera overlay that anchors guidance in the visitor's view of the real world.
- **Unlock location-specific media on confirmed arrival.** Release each checkpoint's images and videos — historical context, testimonials, and narrative clips — only after the geofence has confirmed that the visitor is physically at the location.
- **Drive engagement through gamification.** Sustain exploration with AR Points, milestone completions, optional side quests, post-route knowledge quizzes, digital badges, a community leaderboard, and a store where points are redeemed for tangible perks.
- **Give administrators no-code control.** Allow site administrators to create and manage routes, waypoints, media, quizzes, and users entirely through a dashboard, with engagement analytics generated organically from per-place visitor counts.
- **Remain hardware-free and site-agnostic.** Run entirely in the smartphone browser with no physical installation at the site, so that the same platform can be configured for any campus, museum, or heritage property.

Chapter 4

Market Context

4.1 Target Users

ARadhana serves three user groups. **Students and campus newcomers** are the primary segment: orientation is chaotic, campus geography is unfamiliar, and nothing incentivises exploration; the platform gives them a self-guided AR tour with badges, a peer leaderboard, and tangible perks. **Heritage-site visitors**, domestic and international, form the secondary segment: guides are scarce and cultural context is inaccessible on site; the platform provides a GPS-guided route with story media at each monument and a quiz to reinforce learning. **Site administrators and institutions** are the tertiary segment: guided tours cost staff time, produce no data, and end after orientation day; the platform replaces them with a zero-hardware digital experience and visitor-count analytics.

4.2 Why Nepal

Nepal received 1.147 million international tourists in 2024,[1] holds ten UNESCO World Heritage properties (four cultural sites in the Kathmandu Valley alone),[2] and derives roughly 7–8 % of GDP from tourism. Physical tourism centres are scarce, and UNESCO explicitly observes that few World Heritage sites provide accessible digital visitor resources. The market therefore combines a large addressable audience with a documented absence of digital infrastructure.

4.3 Existing Solutions

Table 4.1 summarises the Nepal-specific alternatives. None combines real-time wayfinding, AR presentation, presence verification, and gamified engagement; one (Saarang) additionally depends on physical QR boards installed and maintained at every site.

Solution	Mechanic	Gap
HoneyGuide Nepal	Curated cultural walking guides	No AR, no navigation loop, no gamification
Nepal Heritage App	Archival site documentation	Database, not a guided journey
GPSmyCity	Generic self-guided GPS city tours	No AR, no engagement mechanics, city-scale only
Kathmandu Durbar App	Paid GPS walking-tour product	Single product, no platform or admin tooling
Lokalee	Digital concierge with one heritage trail	Tours are peripheral to a booking app
Saarang	QR storyboards installed at sites	Physical hardware per site; no navigation

Table 4.1: Nepal-specific competitive landscape.

[1] https://tourism.gov.np/content/83/nepal-tourism-statistics-2024/
[2] https://whc.unesco.org/en/statesparties/np/

4.4 Differentiation

ARadhana addresses every gap in Table 4.1 at once: browser-based location AR with zero physical installation, a live wayfinding loop over WebSocket, geofence-verified presence, a full gamification layer (points, badges, quizzes, side quests, redemption), and an admin GUI with which any institution can configure a new route in minutes. The campus focus is itself a differentiator: the listed competitors target city-scale tourism and leave controlled premises such as campuses and museums unserved.

Chapter 5

Design Process

The product was shaped by one pass through the Stanford d.school design-thinking cycle (empathise, define, ideate, prototype, test). This chapter summarises the decisions that survived into the build.

5.1 Research and Definition

Empathy work combined informal interviews with eight first-year students at Orchid College, observation of a standard orientation day, a review of campus group chats for recurring lost/confused messages, and desk research on Nepal tourism. Recurring findings: students still could not locate basic facilities months after orientation ("I still don't know where the parking area is after 3 months"); the guided tour moved too fast to retain; staff spent 3+ hours per orientation day on tour logistics; and the campus's history had no accessible presentation. The problem was framed as: how might we create a heritage and campus exploration experience that is self-directed, GPS-guided, and story-rich, so that visitors feel oriented and engaged while institutions gain digital visibility with zero hardware.

Three personas guided prioritisation: a first-year student who needs fast orientation and a sense of belonging; an orientation coordinator who needs to reduce manual effort and obtain engagement data without being a developer; and a competitive second-year student motivated by badges, points, and rankings.

5.2 Selected and Rejected Ideas

Ideation converged on GPS-anchored AR overlays, story videos at milestones, physical reward redemption, post-route quizzes, optional side quests, a community page, and an admin waypoint logger — all shipped. Rejected for feasibility or scope: real-time multiplayer racing, QR-code check-ins (no AR value), and creature-collection mechanics. The product was built as a web application rather than a native app so that users enter through a URL with no installation, one codebase covers Android and iOS browsers, and admins use the same origin with role-based routing.

5.3 Validation During the Build

Field testing directly changed the design, as summarised in Table 5.1.

Observation	Design response
5 m arrival threshold too tight; users could not trigger arrival	Default geofence raised to 10 m and made admin-configurable per place
AR video autoplay blocked on iOS	Tap-to-play fallback overlay
Compass inert on first launch (iOS)	Explicit `DeviceOrientationEvent` permission gate behind a user gesture
Quiz felt like a chore after a walk	Capped at five questions; answer-review mode added after the single attempt
Redemption page hard to discover	AR-points balance surfaced on the dashboard header
Indoor demos impossible without GPS	Full demo mode with simulated arrivals

Table 5.1: Field observations and the design responses they produced.

Chapter 6

System Architecture

6.1 Overview

ARadhana is a two-tier web system. The client is a React 19 single-page application (Vite, TypeScript, TailwindCSS v4) that talks to the server over two channels: REST for all resource operations, via RTK Query API slices with declarative cache invalidation, and a WebSocket for the real-time navigation stream. The server is built on `express-file-cluster` (EFC), a file-system-routed Express framework: every file under `src/api/` maps to a URL path, so adding an endpoint means adding a file. The server runs as a Node.js cluster (one primary, two workers sharing a port), persists to MongoDB via Mongoose, queues transactional email through a Redis-backed BullMQ queue, and serves waypoint media from Cloudinary.

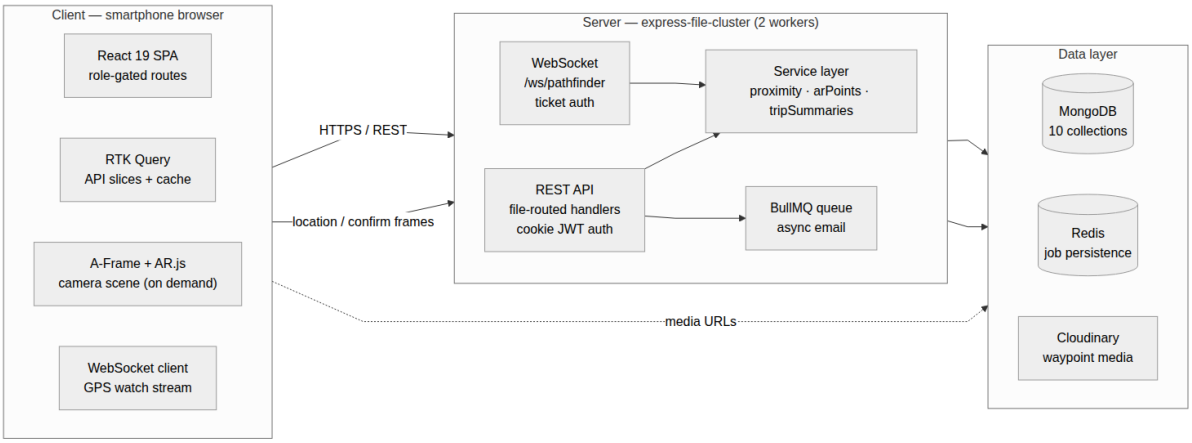

Figure 6.1: System architecture: client, server, and data layers.

6.2 Client

Routing is role-aware: a `ProtectedRoute` component reads the authenticated identity and confines regular users to the exploration experience (dashboard, explore, people, redeem, profile) while admins are routed to a separate dashboard, user management, and the waypoint logger — neither role can reach the other's surface. Data fetching is centralised in RTK Query slices (auth, places, routes, progress, admin, points/quiz), so a mutation such as confirming a visit or redeeming a reward declaratively invalidates exactly the caches that depend on it. The AR layer (A-Frame and AR.js) is deliberately kept outside the React tree and is loaded only when navigation starts; Chapter 9 covers it in detail.

6.3 Server

REST requests pass through cookie-based JWT authentication (`requireAuth` middleware attaches `{id, role, email}` to the request), then reach file-routed handlers that call the models. Business logic that spans endpoints lives in a small service layer: `proximity.ts` (distance, bearing, arrival, confirmation), `arPoints.ts` (award/spend ledger), and `tripSummaries.ts` (per-place progress and badge computation).

Authentication uses short-lived access JWTs and longer-lived refresh JWTs, both in HTTP-only cookies so tokens are invisible to page JavaScript; passwords are stored as bcrypt hashes. The WebSocket cannot carry cookies portably, so it uses a ticket pattern: the client requests a short-lived, single-purpose JWT over authenticated REST and presents it as a query parameter; the server verifies it during the HTTP upgrade, before the socket is accepted.

6.4 Real-Time Location Pipeline

Navigation is a server-authoritative loop:

1. The client seeds an idempotent progress record (`POST /api/user-progress`), fetches a WS ticket, and connects to `/ws/pathfinder`.

2. `navigator.geolocation.watchPosition` streams coordinates; each fix is forwarded as a `location` frame.
3. For every frame the server finds the next unvisited waypoint, computes haversine distance and bearing, and replies with a `progress` frame carrying the distance, bearing, the place's geofence radius (`visit_threshold_meters`, default 10 m, stored per place and read live from the database), and an `arrived` flag.
4. Arrival never auto-completes a checkpoint. The client shows a confirmation prompt; only an explicit `confirm_visit` message — re-validated server-side against the last reported position and the same radius — marks the waypoint visited.
5. Confirmation triggers the gamification side effects (Chapter 8): milestone and side-quest point awards, the permanent milestone log on the user profile, and, when the final checkpoint falls, the place-completion award plus a one-time increment of the place's visitor counter.

Because the threshold, the distance check, and the awards all live server-side, a confirmed visit constitutes evidence of physical presence that a client cannot forge by manipulating its own state.

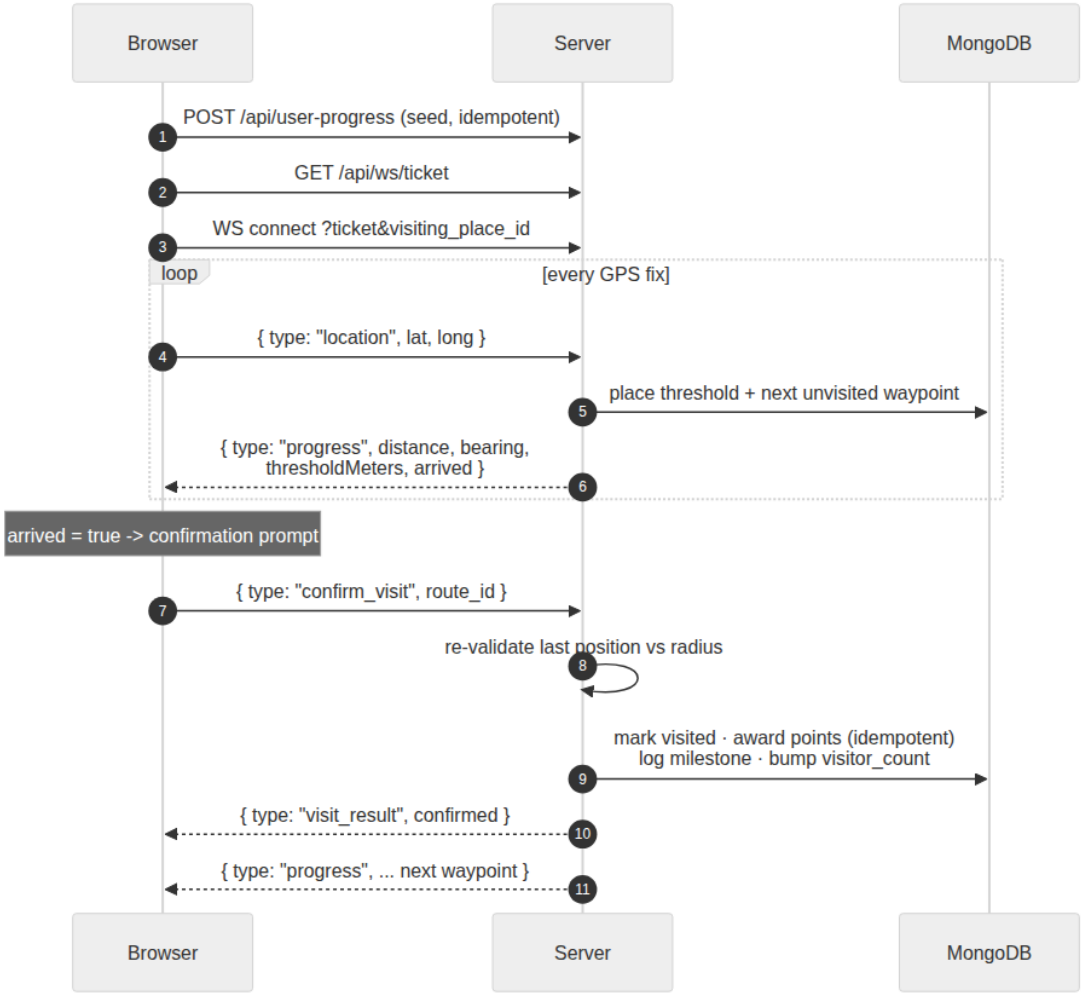

Figure 6.2: Real-time location pipeline: from GPS fix to checkpoint confirmation.

Chapter 7

Database Design

7.1 Entity Overview

MongoDB holds ten collections. Seven carry the product domain:

- **User** — account identity, role, avatar, and a permanent embedded log of earned milestones.
- **VisitingPlace** — a site with central coordinates, badge image, a per-place geofence radius stored as `visit_threshold_meters`, and a denormalised `visitor_count`.
- **VisitingRoutes** — the ordered waypoints of a place; each has GPS coordinates, media, and a type (`start`, `node`, `milestone`, `side_quest`, `end`) that determines behaviour on arrival.
- **UserProgress** — one record per (user, place) holding a visited flag per waypoint; reset-able for revisits.
- **ArPoints** — one append-only points ledger per user.
- **Quiz** / **QuizAttempt** — one five-question quiz per place and one permanent attempt per (user, quiz).

Three support authentication and access control: **Admin** (a separate collection from User), **Session**, and **Role**. Foreign keys are stored as plain string IDs rather than ObjectId references — a deliberate choice that keeps the service layer free of ODM-specific types.

7.2 Key Design Decisions

7.2.1 Separate User and Admin collections

Administrators live in their own collection, so `requireAuth('admin')` queries different data entirely; a role-checking bug in application logic cannot escalate a regular user, because the privilege boundary exists at the data layer.

7.2.2 An append-only points ledger

`ArPoints` is a transaction log, not a mutable balance. Every award appends an entry keyed by `(source, ref_id)`; `awardPoints()` refuses a duplicate key, so re-confirming a checkpoint after a GPS glitch or a progress reset can never farm points. Redemptions append negative entries with unique references (they may legitimately repeat). A running `total` is kept in sync for O(1) balance reads.

7.2.3 Permanent achievements over reset-able progress

`UserProgress` is deliberately disposable — revisiting a place resets it. Achievements therefore live elsewhere: milestones are embedded on the User document at confirmation time, and place completion

is recorded as a `place_complete` ledger entry. Badge computation checks the permanent records as well as live progress, so a badge earned once survives any number of resets.

7.2.4 Denormalised visitor counting

Showing "visited by N explorers" by scanning progress records would cost O(users) per read. Instead, `visitor_count` on the place is incremented exactly once per user, at first completion — the idempotent place-completion award doubles as the uniqueness signal — making the read O(1) at any scale.

7.3 Relationships

Relationship	Cardinality	Foreign key
User → ArPoints	1:1	`ArPoints.user_id`
User → UserProgress	1:N	`UserProgress.user_id`
User → QuizAttempt	1:N	`QuizAttempt.user_id`
VisitingPlace → VisitingRoutes	1:N	`VisitingRoutes.visiting_place_id`
VisitingPlace → UserProgress	1:N	`UserProgress.visiting_place_id`
VisitingPlace → Quiz	1:1	`Quiz.visiting_place_id` (unique)
Quiz → QuizAttempt	1:N	`QuizAttempt.quiz_id`

Table 7.1: Collection relationships.

Field-level schema tables for all collections are given in Appendix B.

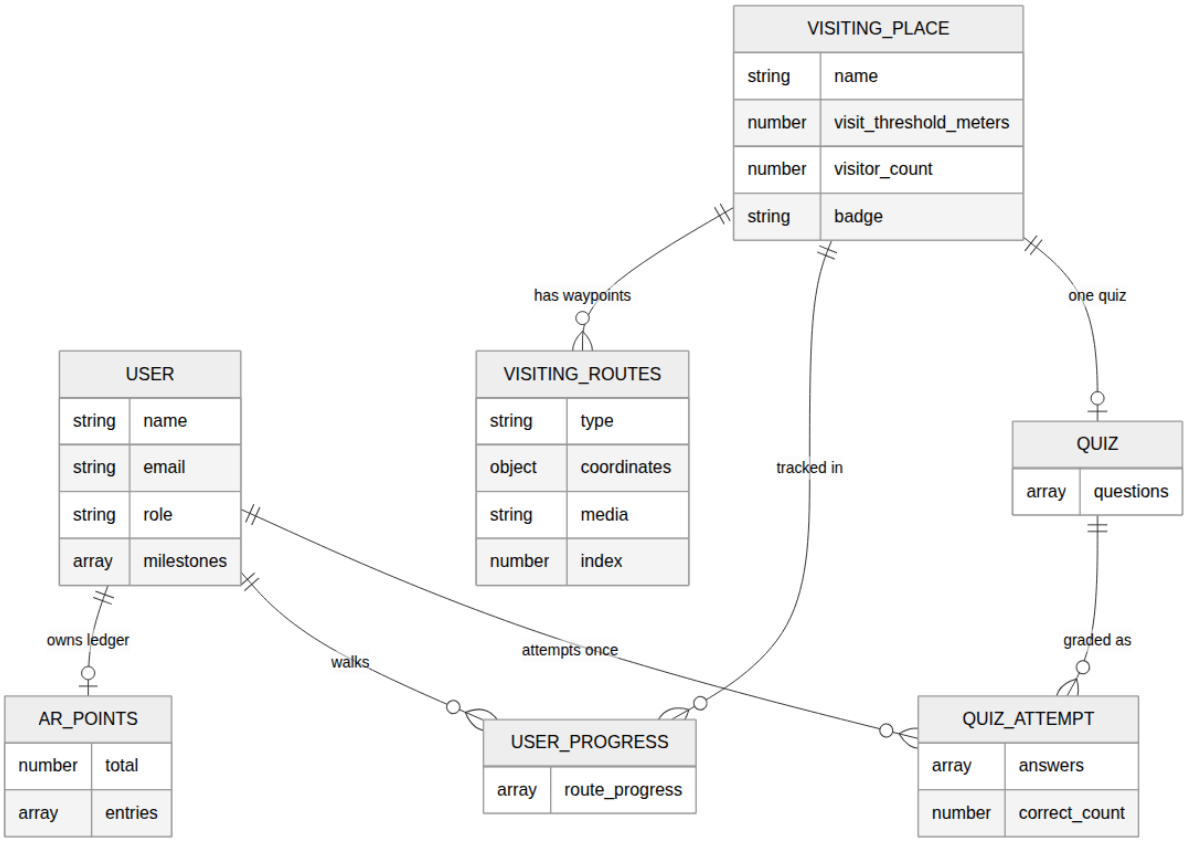

Figure 7.1: Entity–relationship diagram for the seven domain collections.

Chapter 8

Gamification Engine

The gamification layer converts physical exploration into a tracked game loop built from four interlocking systems: AR Points, milestones and badges, side quests, and quizzes. All rewards are persistent (they survive progress resets) and idempotent (repeating an action cannot duplicate a reward).

8.1 AR Points

AR Points are a private virtual currency. The balance appears only on the owner's dashboard and redemption page; it is never included in public profiles, the people directory, or the leaderboard. Table 8.1 lists the earning rules. On the seeded Orchid College route the theoretical maximum is 100 points: two milestones (20), one side quest (25), place completion (20), and a perfect quiz (35).

Action	Ledger source	Points
Visiting a milestone waypoint	`milestone`	10
Completing a side quest (5-minute stay)	`side_quest`	25
Completing every checkpoint of a place	`place_complete`	20
Quiz: 1 / 2 / 3 / 4 / 5 of 5 correct	`quiz`	10 / 20 / 25 / 30 / 35
Redemption	`redeem`	negative (cost)

Table 8.1: AR Points earning and spending rules.

Awards are idempotent by construction: each carries a `(source, ref_id)` key and the ledger rejects duplicates (see the listing in Appendix C), so GPS glitches, re-confirmations, and revisits cannot farm points.

8.2 Milestones and Badges

A milestone is a waypoint whose arrival plays a narrative video; confirming it awards points and appends the milestone permanently to the user's profile. A badge is a place-level award, granted when all of a place's milestone checkpoints are complete (for places without milestones, when the full route is complete). Because badge computation consults the permanent records — profile milestones and the place-completion ledger entry — as well as live progress, badges survive progress resets: a user who revisits a place walks a fresh route but keeps every earned badge.

8.3 Side Quests

A side quest is an optional detour. On arrival its video plays, after which the user may accept or skip. Accepting starts a five-minute countdown during which the user must remain on site; only when the timer completes is the checkpoint confirmed and the 25-point award made. The dwell requirement ensures the reward reflects genuine engagement with the location rather than a tap-and-leave.

8.4 Quizzes

Each place may carry exactly one quiz of five multiple-choice questions. Three rules shape the experience: the quiz is *locked* until the user has completed every checkpoint of the place; each user gets exactly *one attempt*, ever; and correct answers are *stripped* from all API responses to regular users until that attempt is spent. After submission the user sees a scored review — each question with the correct option and their own pick marked — and the entry point changes from "Give quiz" to "View quiz answers".

8.5 Redemption

Points are spent on tangible campus perks defined in a server-side catalog (Table 8.2). Redemption checks the balance, appends a negative ledger entry, and decrements the total; fulfilment is manual

(the student shows the confirmation to staff). Unlike awards, redemptions may repeat, so each receives a unique ledger reference.

Reward	Description	Cost
Canteen tea voucher	A free cup of tea at the campus canteen	30
Orchid sticker pack	Limited-edition campus stickers	60
Table tennis session	30 minutes at the table tennis table	90
Orchid T-shirt	An official Orchid College tee	250

Table 8.2: Rewards catalog.

8.6 Leaderboard and Community

The people section shows a community leaderboard by default, replaced live by search results as the user types (input debounced at 300 ms). It ranks active users by a chosen public metric — milestones, places visited, or side quests completed — with stable tie-breaking across the other two. AR Points are deliberately excluded from the ranking and from public profiles: the competitive surface uses only observable exploration facts, while the currency stays private to its owner.

Chapter 9

AR Pathfinder

The pathfinder combines four real-time data streams — GPS coordinates, device compass heading, WebSocket proximity messages, and camera-based AR rendering — into one navigation experience. Code listings for the formulas referenced here appear in Appendix C.

9.1 Geofencing and Two-Phase Arrival

All proximity checks run server-side using the haversine great-circle formula, accurate to well under a metre at campus distances — far finer than the 3–5 m accuracy of consumer GPS itself. A waypoint counts as reached when the reported position falls within the place's geofence radius. The radius — the `visit_threshold_meters` field on the place — defaults to 10 m rather than being hard-coded: dense environments may need a tighter fence to avoid false arrivals, open spaces a looser one to absorb GPS drift, and administrators can tune it without a deployment.

Arrival is deliberately a two-phase process. Detection — the server reporting `arrived: true` — only raises a confirmation prompt. The checkpoint is marked visited only when the user explicitly confirms, and the server re-validates the last reported position against the same radius before accepting. A

momentary GPS glitch therefore cannot complete a checkpoint, and a confirmed visit is backed by two independent within-radius checks.

9.2 Compass Guidance

The server includes a true-north bearing to the next waypoint in every progress frame. The client subtracts the device's compass heading to obtain the relative bearing that drives the HUD arrow. Two engineering details make this usable. First, raw `DeviceOrientationEvent` readings jitter by several degrees per frame, so headings pass through an exponential moving average (weight 0.15 on the new reading) that removes jitter while still responding to turns within about two seconds. Second, orientation events arrive at up to 60 Hz; routing them through React state would re-render the component tree at that rate, so the arrow and distance text are updated imperatively through refs, bypassing rendering entirely. iOS additionally requires an explicit permission request for orientation data, which is why navigation can only start from a button press (a user gesture).

9.3 AR Camera Layer

The AR view uses A-Frame with AR.js location-based tracking, with the camera feed rendered as a WebGL texture. The `<a-scene>` element is injected directly under `<body>`, outside the React tree — A-Frame requires this to size its fullscreen canvas correctly — and is fully torn down (including stopping webcam tracks) when navigation ends. The vendor scripts are served locally and loaded on demand only when AR mode starts, which pins versions (A-Frame 1.6.0, AR.js 3.4.7), avoids ad-blocker interference, and keeps the initial page load small.

Milestone and side-quest videos play on a plane anchored in world space. Anchoring by GPS coordinates would make the plane jump with satellite drift, so instead the plane spawns four metres in front of the camera, its rotation frozen at spawn (looking around pans off it, as with a real object), while its position is re-glued to the camera's translation every animation frame. The result reads as "placed in the world" yet is immune to GPS jitter.

9.4 Waypoint Types

Type	Behaviour on arrival
`start`	Confirmation prompt; opening image shown
`node`	Turn point; directional image shown on confirmation
`milestone`	Video plays; confirmation awards points and logs the milestone
`side_quest`	Video plays; accept starts the 5-minute dwell timer, skip continues
`end`	Final checkpoint; completion image and trip-complete state

Table 9.1: Waypoint type semantics.

9.5 Graceful Degradation and Demo Mode

The design principle is that the compass arrow is always the fallback. Without HTTPS (camera APIs require a secure context) or if the AR scripts fail to load, navigation continues with the compass HUD and a visible notice; if video autoplay is blocked, a tap-to-play overlay appears; if the compass reports non-absolute headings, a calibration banner is shown. A separate demo mode bypasses GPS entirely: it replays the real route from hardcoded waypoints with a "simulate arrival" control, exercising the complete flow — media, confirmations, rewards UI — indoors, which is essential for stage demonstrations and QA.

9.6 The Seeded Route

The production route walks Orchid College over approximately 400 m (five to seven minutes): a start point, three turn nodes, two milestones with video stories (the parking area and Orchid International College), one side quest (Bishwajit Medical Hall), and an end point, with a 10 m geofence per waypoint and the college badge on completion.

Chapter 10

Technology Stack

10.1 Frontend

Package	Version	Role
`react` / `react-dom`	19.2	UI component framework
`react-router-dom`	7.18	SPA routing with role-gated routes
`@reduxjs/toolkit`	2.12	State + RTK Query data fetching/caching
`tailwindcss` (+ Vite plugin)	4.3	Utility-first styling, CSS-native config
`lucide-react`	1.24	Tree-shakeable SVG icon set
`cartoon-avatar`	1.0	Deterministic avatar image catalog
A-Frame / AR.js (vendored)	1.6.0 / 3.4.7	WebGL AR scene and location tracking
`vite` + `typescript`	8.1 / 6.0	Build tooling, strict typing

Table 10.1: Client stack.

10.2 Backend

Package	Version	Role
`express-file-cluster`	0.3	File-system routing, model layer, clustering, auth
`mongoose`	8.0	MongoDB ODM
`ws`	8.21	Raw WebSocket server for the pathfinder stream
`jose`	6.2	ESM-native JWT signing/verification (incl. WS tickets)
`bcrypt`	5.1	Password hashing (cost 12)
`bullmq` + `nodemailer`	5.0 / 6.9	Redis-queued asynchronous transactional email
`tsx` / `tsup` / `vitest`	—	Script running, production bundling, tests

Table 10.2: Server stack.

10.3 Notable Choices

`ws` over `socket.io`: the pathfinder needs a single bidirectional stream with no rooms, namespaces, or polling fallback; raw WebSocket is a quarter of the bundle size and makes the ticket-based upgrade authentication straightforward. **RTK Query over React Query**: it ships inside Redux Toolkit at zero extra dependency and makes cross-feature cache invalidation (confirm a visit → refresh progress, points, and leaderboard) declarative. **`jose` over `jsonwebtoken`**: ESM-native and actively maintained, matching the fully `"type": "module"` server. **BullMQ for email**: SMTP delivery takes 200–2000 ms; queueing it keeps registration and password-reset responses immediate. **Vendored AR libraries**: version pinning and immunity to ad-blockers and CDN failures, loaded on demand so the initial bundle stays small.

Chapter 11

Results

11.1 Delivered Features

The hackathon build delivers a complete vertical slice, working end to end:

- **Exploration.** Route selection, live compass HUD with distance readout, AR camera overlay with world-anchored story videos, geofence-verified two-phase checkpoint confirmation, media unlocks per waypoint type, side-quest dwell timer, revisit with progress reset, and a GPS-free demo mode.
- **Gamification.** Private AR Points ledger with idempotent awards, permanent milestone log and badges, per-place quizzes with completion lock, single attempt, and answer review, and a rewards catalog with confirm-before-spend redemption.

- **Community.** Public profiles, a people directory with debounced search, and a leaderboard ranked by milestones, places, or side quests with AR Points kept private.
- **Administration.** A role-separated admin experience: statistics dashboard with 30-day signup and milestone trends, user management (verify, suspend, reactivate, promote, delete), the waypoint logger for route authoring, and per-place quiz and geofence-radius configuration.

11.2 Field Test

The complete loop was validated on the seeded Orchid College route (eight waypoints, ∼400 m) and a second short route at the hackathon venue. Real users registered, walked the routes, and confirmed checkpoints: arrival prompts fired inside the 10 m geofence, confirmations persisted across sessions, milestone videos played anchored in the camera view, the side-quest timer enforced the five-minute stay, badges and completion points were awarded exactly once per user, quizzes unlocked only after completion, and redeemed rewards appeared as negative ledger entries. The place visitor counters advanced once per unique completer, and the admin dashboard reflected the activity without any manual instrumentation.

11.3 Screenshots

Screenshots of the main surfaces (explore, AR navigation, quiz, redemption, leaderboard, and admin dashboard) are to be captured from the deployed build and inserted here.

Chapter 12

Limitations and Future Work

12.1 Known Limitations

- **GPS accuracy.** Consumer GPS is accurate to roughly 3–15 m and degrades near buildings. Against a 10 m geofence this can require a user standing at a checkpoint to wait for the fix to settle before the arrival prompt fires; the per-place radius mitigates but does not eliminate this.
- **Secure-context requirement.** Camera and precise geolocation APIs are available only over HTTPS (or localhost); on an insecure origin the system degrades to the compass-only HUD.
- **External media dependencies.** Waypoint media is served from Cloudinary and avatar images from a public CDN; both require connectivity, and there is no offline mode.
- **Compass variability.** Device magnetometers differ in quality; non-absolute headings trigger a calibration banner and may need a figure-eight gesture.

- **Permanent quiz attempts.** An attempt is permanent even if an administrator later edits the quiz content.
- **Single language.** The interface and content are English-only.

12.2 Future Work

Planned directions, in rough priority order: additional heritage routes (the Kathmandu Valley UNESCO properties are natural candidates); multi-language content, which matters more for heritage visitors than for campus use; an in-dashboard quiz authoring interface (quizzes are currently managed through the API and seed scripts); richer analytics such as dwell time and per-waypoint drop-off; offline route bundles for areas with poor connectivity; native app packaging for tighter sensor access; and an accessibility pass across the exploration flow.

Chapter 13

Conclusion

The problem this project set out to address is that guidance for visitors in Nepal ends at the entrance: inside a heritage site or a campus there is no structured navigation, no storytelling at the places where it matters, no incentive to explore, and no way for an institution to verify that anyone was actually there.

ARadhana answers with a complete, working system rather than a concept. A browser-based AR pathfinder guides visitors waypoint by waypoint; a server-authoritative geofence with two-phase confirmation makes every recorded visit evidence of physical presence; a gamification engine — points, milestones, badges, side quests, quizzes, and redeemable rewards — gives exploration a reason to continue and to repeat; and a role-separated admin surface lets a non-technical institution configure routes, media, quizzes, and geofence radii without touching code.

The build was validated where it counts: on the ground. Real users walked the seeded Orchid College route during the hackathon, triggered geofenced arrivals, watched story media anchored in the camera view, earned badges and points exactly once each, and appeared on the leaderboard — while the admin dashboard accumulated engagement data as a side effect of ordinary use. Because every site-specific element is configuration rather than code, the same platform that guided a campus walk can be pointed at any museum or heritage property with an admin account and a set of coordinates. The prototype thus demonstrates the full loop it promised: verified presence, engaging interpretation, and measurable exploration, delivered as software alone.

References

[1] Ministry of Culture, Tourism and Civil Aviation, Nepal, "Nepal tourism statistics 2024." https://tourism.gov.np/content/83/nepal-tourism-statistics-2024/, 2024. Accessed: 2026-07-11.

[2] UNESCO World Heritage Centre, "World heritage sustainable tourism toolkit, guide 5: Communication with visitors." https://whc.unesco.org/en/sustainabletourismtoolkit/guide5/. Accessed: 2026-07-11.

[3] UN Tourism (UNWTO) and UNESCO, "Third unwto/unesco world conference on tourism and culture: Concept note." https://webunwto.s3-eu-west-1.amazonaws.com/imported_images/50679/unwto_unesco_istanbul_conf_concept_note_22.11.2018_en.pdf, 2018. Istanbul, Accessed: 2026-07-11.

[4] AND Journal, "Tourism: Importance, prospects and challenges with special reference to nepal." https://andjournal.in/2018/07/13/tourism-importance-prospects-and-challenges-with-special-reference-to-nepal/, 2018. Accessed: 2026-07-11.

[5] A. Umanets, A. Ferreira, and N. Leite, "Guideme — a tourist guide with a recommender system and social interaction." https://arxiv.org/abs/1402.1243, 2014. arXiv:1402.1243.

[6] AR.js Organization, "Ar.js — augmented reality on the web." https://ar-js-org.github.io/AR.js-Docs/. Version 3.4.7, Accessed: 2026-07-11.

[7] Supermedium, "A-frame: Web framework for building virtual reality experiences." https://aframe.io. Version 1.6.0, Accessed: 2026-07-11.

[8] R. W. Sinnott, "Virtues of the haversine," *Sky and Telescope*, vol. 68, no. 2, p. 159, 1984.

Chapter A

API Reference

All REST endpoints live under the server base URL. Authentication uses HTTP-only cookie JWTs (access + refresh); the WebSocket authenticates with a short-lived ticket obtained over authenticated REST. "User" means any signed-in account; "admin" requires the admin role.

Table A.1: REST API endpoints.

Endpoint	Auth	Description
Authentication		
`POST /api/auth/register`	—	Create an account
`POST /api/auth/login`	—	Log in; sets auth cookies
`POST /api/auth/logout`	user	Clear cookies, revoke refresh token
`GET /api/auth/me`	user	Authenticated identity
`POST /api/auth/refresh`	—	Rotate the access token
`POST /api/auth/verify-email`	—	Verify email by token
`POST /api/auth/forgot-password`	—	Send reset email
`POST /api/auth/reset-password`	—	Reset password by token
`POST /api/auth/change-password`	user	Change own password
`GET /api/ws/ticket`	user	Short-lived WebSocket ticket
Visiting places and routes		
`GET /api/visiting-places`	user	List places (incl. geofence radius, visitor count)
`POST /api/visiting-places`	admin	Create a place
`PUT /api/visiting-places/:id`	admin	Update a place (incl. geofence radius)
`DELETE /api/visiting-places/:id`	admin	Delete place, cascade routes/progress
`GET /api/visiting-routes`	user	Waypoints of a place, ordered
`POST /api/visiting-routes`	admin	Create a waypoint
`PUT /api/visiting-routes/:id`	admin	Update a waypoint
`DELETE /api/visiting-routes/:id`	admin	Delete a waypoint
Progress, quiz, points		
`GET /api/user-progress`	user	Own progress for a place
`POST /api/user-progress`	user	Seed progress (idempotent)
`GET /api/user-progress/summary`	user	Trips, milestones, badge status
`POST /api/user-progress/reset`	user	Reset a route for a revisit
`GET /api/visiting-places/:id/quiz`	user	Quiz (answers stripped; locked until complete)
`POST /api/visiting-places/:id/quiz/attempt`	user	Submit the single attempt
`PUT /api/visiting-places/:id/quiz`	admin	Create/replace the 5-question quiz
`GET /api/user/points`	user	Private balance, ledger, rewards catalog
`POST /api/user/points/redeem`	user	Redeem a reward
Community		
`GET /api/users/search`	user	Search members by name/email (public card data)
`GET /api/users/leaderboard`	user	Ranked by milestones / places / side quests
`GET /api/users/:id`	user	Public profile (no email, no points)
`GET /api/user/profile`	user	Own profile
`PUT /api/user/profile`	user	Update name, email, avatar
Administration		
`GET /api/admin/dashboard`	admin	Platform statistics
`GET /api/admin/analytics`	admin	Trends and per-place completion

Endpoint	Auth	Description
`GET /api/admin/users`	admin	List users (paginated)
`PUT /api/admin/users/:id`	admin	Update role/identity/status
`POST /api/admin/users/:id/suspend`	admin	Suspend account
`POST /api/admin/users/:id/activate`	admin	Reactivate account
`POST /api/admin/users/:id/verify`	admin	Mark email verified
`DELETE /api/admin/users/:id`	admin	Delete account
`GET /api/health`	—	Liveness check

A.1 WebSocket Protocol

The pathfinder channel is `/ws/pathfinder?ticket=<JWT>&visiting_place_id=<id>`. The client sends `location` frames (`{type, lat, long}`) as geolocation fixes arrive and `confirm_visit` frames (`{type, route_id}`) when the user confirms. The server replies to every location with a `progress` frame — next waypoint, distance, bearing, the place's geofence radius, and `arrived`/`allVisited` flags — and to every confirmation with a `visit_result` frame carrying `confirmed` and, on rejection, a human-readable `reason`.

Chapter B

Database Schema Reference

Table B.1: User schema

Field	Type	Notes
`name`	string	Display name
`email`	string	Unique; login identity
`password`	string	bcrypt hash
`role`	string	`user` (default) or `admin`
`milestones[]`	array	Permanent `{name, earned_at}` log; survives resets
`avatar`	string	Image URL
`isVerified` / `isActive`	boolean	Email verified; soft-disable flag
`*Token(+Expiry)`	string/Date	Verify, reset, and refresh token state

Table B.2: VisitingPlace schema

Field	Type	Notes
`name` / `description`	string	Card display
`lat` / `long`	string	Central coordinate
`badge`	string	Badge image URL awarded on completion
`visit_threshold_meters`	number	Geofence radius; required, default 10
`visitor_count`	number	Unique completers; bumped once per first completion

Table B.3: VisitingRoutes schema

Field	Type	Notes
`visiting_place_id`	string	Owning place
`name` / `description`	string	HUD and arrival-card text
`type`	enum	`start`, `node`, `milestone`, `side_quest`, `end`
`coordinates.{lat,long}`	string	Waypoint GPS position
`media`	string	Image or video URL
`index`	number	0-based route ordering

Table B.4: UserProgress schema

Field	Type	Notes
`user_id` / `visiting_place_id`	string	One record per pair
`route_progress[]`	array	`{route_id, route_index, visited}` per waypoint

Table B.5: ArPoints schema

Field	Type	Notes
`user_id`	string	Unique; one ledger per user
`total`	number	Running balance (O(1) reads)
`entries[].source`	enum	`quiz`, `side_quest`, `milestone`, `place_complete`, `redeem`
`entries[].ref_id`	string	Idempotency key with source
`entries[].points`	number	Negative for redemptions
`entries[].label` / `earned_at`	string	Display text; ISO timestamp

Table B.6: Quiz and QuizAttempt schema

Field	Type	Notes
`Quiz.visiting_place_id`	string	Unique — one quiz per place
`Quiz.questions[]`	array	`{question, options[], correct_index}`; exactly five; answers never sent to users
`QuizAttempt.user_id` / `quiz_id`	string	One permanent attempt per pair
`QuizAttempt.answers[]`	number[]	Picked option per question (enables answer review)
`QuizAttempt.correct_count` / `points`	number	Score and award

Table B.7: Admin, Session, Role schema

Collection	Type	Description
`Admin`	record	Mirrors `User` (name, email, password, flags) with a `permissions[]` array.
`Session`	record	Records `{userId, token, ip, userAgent, expiresAt, isActive}` for multi-device management.
`Role`	record	Defines named permission sets `{name, description, permissions[]}`.

Chapter C

Selected Code Listings

Haversine Distance (server-side proximity)

```
export function haversineDistanceMeters(lat1, long1, lat2, long2): number {
  const R = 6371000; // Earth radius in metres
  const phi1 = toRad(lat1);
  const phi2 = toRad(lat2);
  const dPhi = toRad(lat2 - lat1);
  const dLambda = toRad(long2 - long1);
  const a = Math.sin(dPhi / 2) ** 2 +
            Math.cos(phi1) * Math.cos(phi2) * Math.sin(dLambda / 2) ** 2;
  return R * 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1 - a));
}
```

Listing C.1: Great-circle distance in metres (server/src/lib/proximity.ts).

Bearing to the Next Waypoint

```
export function bearingDegrees(lat1, long1, lat2, long2): number {
  const dLng = toRad(long2 - long1);
  const phi1 = toRad(lat1);
  const phi2 = toRad(lat2);
  const x = Math.sin(dLng) * Math.cos(phi2);
  const y = Math.cos(phi1) * Math.sin(phi2) -
            Math.sin(phi1) * Math.cos(phi2) * Math.cos(dLng);
  return ((Math.atan2(x, y) * 180) / Math.PI + 360) % 360;
}
```

Listing C.2: True-north bearing driving the HUD arrow.

Compass Heading Smoothing (client)

```
let diff = rawHeading - prev;
diff = (((diff + 180) % 360) + 360) % 360 - 180; // shortest path
smoothedHeading = (prev + 0.15 * diff + 360) % 360;
```

Listing C.3: Exponential moving average over the shortest angular path.

Idempotent Point Awards

```
const alreadyAwarded = ledger.entries.some(
  (e) => e.source === entry.source && e.ref_id === entry.ref_id,
);
if (alreadyAwarded) return null; // re-confirming can never farm points
```

Listing C.4: The (source, ref_id) ledger guard (server/src/lib/arPoints.ts).

AR Scene Configuration

```
<a-scene
  vr-mode-ui="enabled: false"
  arjs="sourceType: webcam; videoTexture: true; debugUIEnabled: false"
  renderer="antialias: true; alpha: true"
  loading-screen="enabled: false">
  <a-camera
    id="camera"
    gps-new-camera="gpsMinDistance: 3"
    arjs-device-orientation-controls="smoothingFactor: 0.8"
    look-controls-enabled="false">
```

```
  </a-camera>
</a-scene>
```

Listing C.5: A-Frame/AR.js location-based scene (injected under <body>).

World-Anchored Video (GPS-jitter immunity)

```
const follow = () => {
  camObj.getWorldPosition(camPos);
  obj.position.set(camPos.x + offset.x,
                   camPos.y + offset.y,
                   camPos.z + offset.z);
  rafId = requestAnimationFrame(follow);
};
rafId = requestAnimationFrame(follow);
```

Listing C.6: Rotation frozen at spawn; position re-glued to the camera each frame.